

L11 Colab 教学数据分析进阶练习题集

这两道题继续在 **Google Colab** 上完成。操作步骤：

1. 打开 [Google Colab](#)
2. 新建一个 Notebook
3. 把数据文件上传到 Colab（点击左侧文件夹图标，然后上传）
4. 在代码格里写代码，运行看结果

完成后把你的 **Colab 链接** 放到 `submission` 文件夹下。

这次的数据不再是天气，也不只是普通成绩表，而是更贴近 UNSW Python 教学场景的：**Python 学习日志数据**。

想象 COMP1010 / INFO1110 这种 Python 课程里，学生每周会做不同主题的编程练习，例如 variables、loops、functions、file I/O、pandas、groupby。系统会记录每次提交的测试通过情况、代码行数、花费时间和提交时间。

数据文件叫 `python_learning_log.csv`，一行代表一名学生在某一周、某个 Python 主题上的一次练习记录。

数据文件有 8 列：

列名	含义
<code>student_id</code>	学生编号
<code>week</code>	第几周
<code>topic</code>	Python 学习主题
<code>lines_of_code</code>	代码行数
<code>tests_passed</code>	通过的测试数
<code>tests_total</code>	总测试数
<code>minutes_spent</code>	完成练习花费的分钟数
<code>submission_hour</code>	提交时间，0 到 23

这节课重点不是只算平均值，而是学习更真实的数据分析步骤：

1. 从原始列计算一个新的指标
2. 再按组统计这个指标

3. 用多个指标合成一个简单的“风险指数”
4. 根据结果排序，找出需要关注的学生或主题

Exercise 1: 各 Python 主题平均测试通过率

计算每个 Python 学习主题的平均测试通过率，按主题名字母顺序输出。

每一条记录的测试通过率是：

```
pass_rate = tests_passed / tests_total
```

例如某条记录 `tests_passed = 7`，`tests_total = 8`，那么通过率就是 `7 / 8 = 0.875`，也就是 `87.5%`。

这道题要求用两种方法完成：

1. **Method 1:** 使用 Python 原生方法 `with open` 读取
2. **Method 2:** Use package，也就是用 `pandas` 的简洁语法实现

两种方法的输出必须完全一样。先用 Method 1 理清楚：读取文件、跳过标题行、逐行拆字段、计算新指标、用字典分组、手动平均。再用 `pandas` 做同样的事情。

Method 1: 使用 Python 原生方法 `with open` 读取

要求：

1. 用 `with open("python_learning_log.csv") as f` 打开文件
2. 用 `next(f)` 跳过第一行标题
3. 每一行用 `strip()` 去掉末尾换行，再用 `split(",")` 拆字段
4. 取出 `topic`（第 3 列，下标 2）
5. 取出 `tests_passed`（第 5 列，下标 4）和 `tests_total`（第 6 列，下标 5）
6. 把 `tests_passed` 和 `tests_total` 转成 `float`
7. 计算 `pass_rate = tests_passed / tests_total`
8. 用一个字典，key 是 `topic`，value 是该 `topic` 所有通过率的列表
9. 按 `topic` 字母顺序输出：`topic : 平均通过率 %`
10. 平均通过率用百分比显示，保留 1 位小数

测试代码 + 预期输出

```
# dictionaries : 76.7 %
# file_io : 77.5 %
# functions : 76.0 %
# lists : 80.0 %
# loops : 77.5 %
# pandas_groupby : 70.0 %
# pandas_intro : 76.0 %
# variables : 88.0 %
```

代码框架

```
pass_rates_by_topic = {}

with open("python_learning_log.csv") as f:
    next(f) # 跳过标题行
    for row in f:
        values = row.strip().split(",")

        # 取出 topic、tests_passed、tests_total
        topic = None
        tests_passed = None
        tests_total = None

        # 计算通过率
        pass_rate = None

        # 如果这个 topic 第一次出现，先创建空列表
        # 然后把 pass_rate 追加进去
        pass

# 按 topic 字母顺序输出
for topic in sorted(pass_rates_by_topic):
    rates = pass_rates_by_topic[topic]
    avg_rate = sum(rates) / len(rates)
    print(topic, ":", round(avg_rate * 100, 1), "%")
```

Method 2: Use package, 用 pandas 简洁语法实现

同样的结果，用 pandas 可以更短：`read_csv` → 创建新列 → `groupby` → `.mean()`。

要求：

1. 导入 `pandas as pd`
2. 用 `pd.read_csv()` 读取 `python_learning_log.csv`

3. 新建一列 `pass_rate`，值为 `tests_passed / tests_total`
4. 用 `topic` 分组
5. 对 `pass_rate` 求 `.mean()`
6. 按 `topic` 字母顺序输出：`topic : 平均通过率 %`
7. 平均通过率用百分比显示，保留 1 位小数

测试代码 + 预期输出

```
# dictionaries : 76.7 %  
# file_io : 77.5 %  
# functions : 76.0 %  
# lists : 80.0 %  
# loops : 77.5 %  
# pandas_groupby : 70.0 %  
# pandas_intro : 76.0 %  
# variables : 88.0 %
```

代码框架

```
import pandas as pd  
  
df = pd.read_csv("python_learning_log.csv")  
  
# 创建通过率新列  
df["pass_rate"] = None  
  
# 按 topic 分组，对 pass_rate 求平均值  
avg_pass_rate = None  
  
# 按 topic 字母顺序输出  
for topic in sorted(avg_pass_rate.index):  
    print(topic, ":", round(float(avg_pass_rate[topic]) * 100, 1), "%")
```

对比两种方法

	Method 1 <code>with open</code>	Method 2 <code>pandas</code>
读文件	手动 <code>open</code> 、 <code>next(f)</code> 跳标题、 <code>split</code> 拆字段	<code>pd.read_csv()</code>
新指标	逐行计算 <code>tests_passed / tests_total</code>	整列计算 <code>df["tests_passed"] / df["tests_total"]</code>
分组	自己用字典收集列表	<code>groupby("topic")</code>
求平均	<code>sum(list) / len(list)</code>	<code>.mean()</code>
输出	手动乘以 100 变成百分比	同样手动乘以 100

这道题比前面更进一步：不是直接统计原始列，而是先从原始数据计算出一个更有意义的教学指标。

Exercise 2: 学习风险指数排名 ★★★★★

现在我们要做一个更像真实教学分析的问题：根据学生的编程练习表现，找出哪些学生可能需要更多帮助。

我们定义三个指标：

1. 测试通过率

```
pass_rate = tests_passed / tests_total
```

通过率越低，风险越高。

2. 深夜提交

如果提交时间在晚上 9 点之后，或者凌晨 2 点之前，就算深夜提交：

```
late_submission = 1 if submission_hour >= 21 or submission_hour <= 2 else 0
```

深夜提交不一定代表不好，但在教学数据里，它可能提示学生在赶 deadline 或学习节奏有压力。

3. 学习风险指数

我们用一个小公式合成风险指数：

```
risk_score = (1 - pass_rate) * 60 + (minutes_spent / 80) * 25 +  
late_submission * 15
```

解释：

- $(1 - \text{pass_rate}) * 60$ ：测试没通过的比例越高，风险越高
- $(\text{minutes_spent} / 80) * 25$ ：花费时间越长，可能说明题目卡住了
- $\text{late_submission} * 15$ ：深夜提交加一点风险权重

你的任务是：对每个学生，计算他们所有记录的平均 `risk_score`，然后按风险指数从高到低输出。

这题比前面更高深，因为它不只是 `groupby`，而是先做特征工程：从原始列构造 `pass_rate`、`late_submission`、`risk_score` 三个新列，再做分组排序。

要求：

1. 导入 `pandas as pd`
2. 读取 `python_learning_log.csv`
3. 新建 `pass_rate` 列：`tests_passed / tests_total`
4. 新建 `late_submission` 列：如果 `submission_hour >= 21` 或 `submission_hour <= 2`，值为 1，否则为 0
5. 新建 `risk_score` 列，公式如下：

```
risk_score = (1 - pass_rate) * 60 + (minutes_spent / 80) * 25 +  
late_submission * 15
```

6. 用 `student_id` 分组，对 `risk_score` 求平均值
7. 用 `sort_values(ascending=False)` 按风险指数从高到低排序
8. 输出：`student_id : 平均风险指数`
9. 平均风险指数保留 1 位小数

测试代码 + 预期输出

```
# z5400004 : 65.0  
# z5400002 : 49.5  
# z5400005 : 19.0  
# z5400001 : 15.6  
# z5400003 : 8.2
```

代码框架

```
import pandas as pd

df = pd.read_csv("python_learning_log.csv")

# 计算测试通过率
df["pass_rate"] = None

# 判断是否深夜提交
# 提示：可以用 .apply(lambda h: ...)
df["late_submission"] = None

# 计算学习风险指数
df["risk_score"] = None

# 按 student_id 分组，对 risk_score 求平均值
avg_risk_by_student = None

# 按风险指数从高到低排序
sorted_risk = None

# 输出结果
for student_id, score in sorted_risk.items():
    print(student_id, ":", round(float(score), 1))
```

这题在练什么

1. 特征工程：从原始数据构造更有意义的新指标
2. `.apply(lambda h: ...)`：对一整列应用一个判断函数
3. 多列组合计算：一个公式里同时使用 `pass_rate`、`minutes_spent`、`late_submission`
4. `groupby().mean()`：对每个学生聚合多条记录
5. `sort_values(ascending=False)`：按统计结果排序
6. 教学数据分析不只是算平均分，也可以帮助老师发现哪些学生可能需要及时支持

提交说明

完成后，在 `submission` 文件夹下新建一个文件（比如 `你的名字_L11.txt`），把你的 Colab Notebook 链接粘贴进去。

Exercise 3 : LeetCode 练习：滑动窗口

完成 滑动窗口 的阅读：

<https://labuladong.online/zh/algo/essential-technique/sliding-window-framework/>

在 LeetCode 上完成以下 练习，并截图提交：

76. 最小覆盖子串

Game start

参考 readme.md 完成day3 基础移动

最实用的顺序就是这个：

1. 先让 `parse()` 能读 `board_simple.txt`
2. 再让 `grid_to_string()` 能打印地图
3. 再让玩家可以用 `d / a / w / s` 移动
4. 再加撞墙判断
5. 再加 `W` 水桶
6. 再加 `F` 火
7. 再加 1-9 传送门
8. 最后加胜利、失败、退出、非法输入